Cryptography Fundamentals

IE3122: Network Security | Week 12

Symmetric, asymmetric, block, stream, and how real protocols combine them

Today's Agenda

- Foundations: what cryptography provides, its vocabulary, and Kerckhoffs's principle
- Classical ciphers: Caesar and Vigenere, and exactly why they fail
- Symmetric encryption and the key distribution problem
- Block ciphers versus stream ciphers: modes, algorithms, strengths, weaknesses, and when to choose each
- Asymmetric encryption: RSA, Diffie-Hellman, and elliptic curve cryptography
- Symmetric versus asymmetric, and how hybrid encryption combines them in TLS and IPsec

Learning Outcomes for Today

LO3

- Analyse security requirements and recommend appropriate technologies.

LO4

- Evaluate different security technologies, compare them, and identify appropriate use cases.
- *By the end of this session you should be able to explain how each cipher family works, and recommend the right one for a given scenario.*

Foundations

What cryptography is for, and the vocabulary we need

Why Cryptography Exists

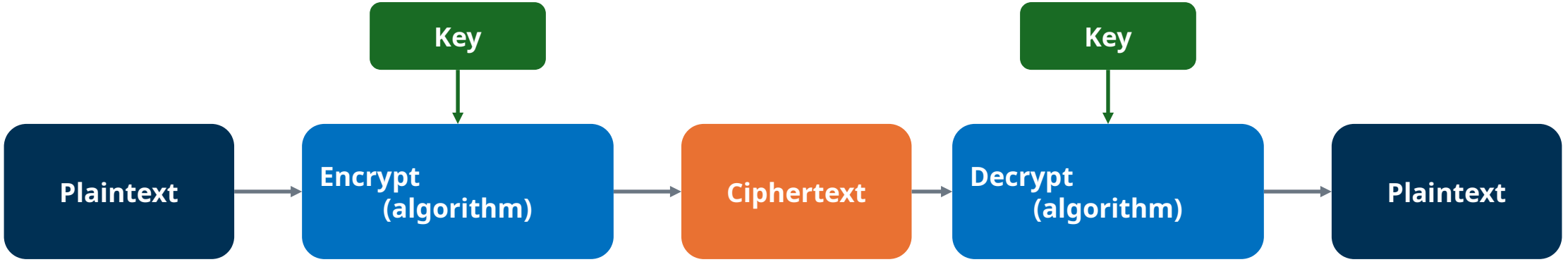
- Recall Week 9: the internet is an untrusted network. Any device along the path can read, copy, or modify the traffic passing through it.
- We usually cannot control the path, so we protect the data itself. Cryptography is the mathematics of making data unreadable, tamper-evident, and attributable, even when an attacker can see every packet.
- Almost every defence in this module depends on it: VPN tunnels (Week 9), AAA credential protection (Week 10), and secure routing authentication (Week 5) are all cryptography applied to a networking problem.
- Today and next we build the toolbox. This session covers encryption, which provides confidentiality. The next deck covers hashing, signatures, and certificates.

The Four Security Services

Security Service	The Question It Answers	Cryptographic Tool	Covered In
Confidentiality	Can anyone else read this?	Symmetric and asymmetric encryption	This deck
Integrity	Was this changed in transit?	Hash functions, MACs	Next deck
Authentication	Who really sent this?	MACs, digital signatures, certificates	Next deck
Non-repudiation	Can the sender deny it later?	Digital signatures	Next deck

Non-repudiation is the same concept met in Week 2, and the goal that command accounting supported in Week 10.

Terminology



- Plaintext is the readable original; ciphertext is the scrambled result. The algorithm (cipher) is the method; the key is the secret value that controls it.
- Encryption turns plaintext into ciphertext. Decryption reverses it. Cryptanalysis is the study of breaking this without the key.

Kerckhoffs's Principle

- A cryptosystem should remain secure even if everything about it, except the key, is public knowledge.
- Why this matters: secret algorithms get leaked, reverse engineered, or simply turn out to be weak. Public algorithms get attacked by thousands of researchers, and the survivors earn trust. AES was selected through an open public competition for exactly this reason.
- The practical consequence: never design your own cipher and never rely on obscurity. Use standard, well-reviewed algorithms, and put all of your effort into protecting the key.
- The warning examples are the proprietary ciphers such as GSM's A5/1 and WEP's RC4 usage, which were weakened or broken once they were examined openly. We meet both later today.

Callback: Week 9 Was Built From These Primitives

- Recall the IPsec framework table: confidentiality (DES, 3DES, AES), integrity (MD5, SHA), authentication (pre-shared key or RSA signatures), and a Diffie-Hellman group. Every row is a cryptographic primitive.
- Recall the SSL VPN building blocks: key exchange, authentication, and bulk encryption. The bulk encryption row listed RC4 and ChaCha20 alongside DES, 3DES, and AES.
- Those two groups are exactly the two families we examine today: stream ciphers (RC4, ChaCha20) and block ciphers (DES, 3DES, AES).
- By the end of the next deck you will be able to explain every cell of both tables, not just recognise the names.

Classical Ciphers

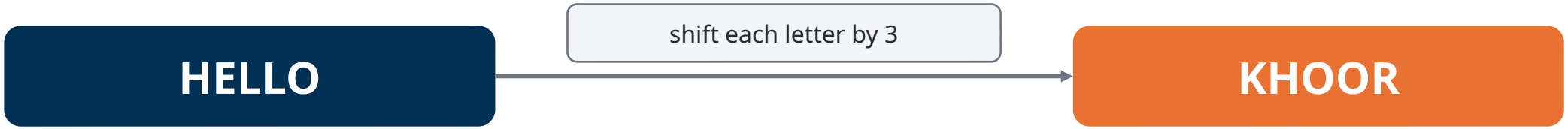
Simple ideas, and why they fail

Substitution and Transposition

- Substitution: replace each symbol with a different symbol. The letters keep their positions but change their identity. Caesar and Vigenere are both substitution ciphers.
- Transposition: rearrange the positions of the symbols without changing them. For example, write a message in rows of five and read it out column by column.
- Every modern cipher combines both ideas many times over, which is why we study the simple versions first. Substitution hides which symbol is which; transposition spreads the message out.
- Classical ciphers are not used for real security today. Their value is that their failure is easy to see, and the failures teach the design goals of modern ciphers.

Caesar Cipher

- Mechanism: shift every letter forward by a fixed number of places in the alphabet. The shift amount is the key. Julius Caesar reportedly used a shift of 3.
- Worked example with a key of 3: A becomes D, B becomes E, and so on, wrapping from Z back to A.



- Decryption shifts backwards by the same amount. Sender and receiver must share the key (the shift), which makes this a symmetric cipher.
- Strength: trivial to use by hand. Weakness: there are only 25 possible keys, so an attacker simply tries all of them.

Vigenere Cipher

- Mechanism: use a keyword instead of a single shift. Repeat the keyword along the message, and shift each letter by the value of the keyword letter above it (A is 0, B is 1, and so on).
- Worked example with the keyword LEMON and the message ATTACKATDAWN:

Message	A	T	T	A	C	K	A	T	D	A	W	N
Key (repeated)	L	E	M	O	N	L	E	M	O	N	L	E
Ciphertext	L	X	F	O	P	V	E	F	R	N	H	R

- Notice that the letter A becomes L, then O, then E, depending on the key letter. The same plaintext letter no longer always produces the same ciphertext letter, which defeats simple frequency counting.
- Seen as a keystream: the repeated keyword acts as a short repeating keystream added to the message. This is a useful bridge to stream ciphers later today, though it is a conceptual framing rather than the formal definition.

Why Classical Ciphers Fail

- Tiny key space: Caesar has only 25 usable keys. A computer, or a patient human, tries every one instantly. This is a brute force attack.
- Frequency analysis: in English the letter E is the most common, followed by T and A. In a Caesar ciphertext the most frequent letter is almost certainly the encrypted E, which reveals the shift immediately.
- Vigenere flattens the frequencies, but it still leaks structure. If the keyword length is found (the Kasiski examination looks for repeated ciphertext fragments), the message splits into several ordinary Caesar ciphers, each broken by frequency analysis.
- The lesson: the ciphertext must not reveal any statistical pattern of the plaintext, and the key space must be far too large to search. Both ideas shape every modern cipher.

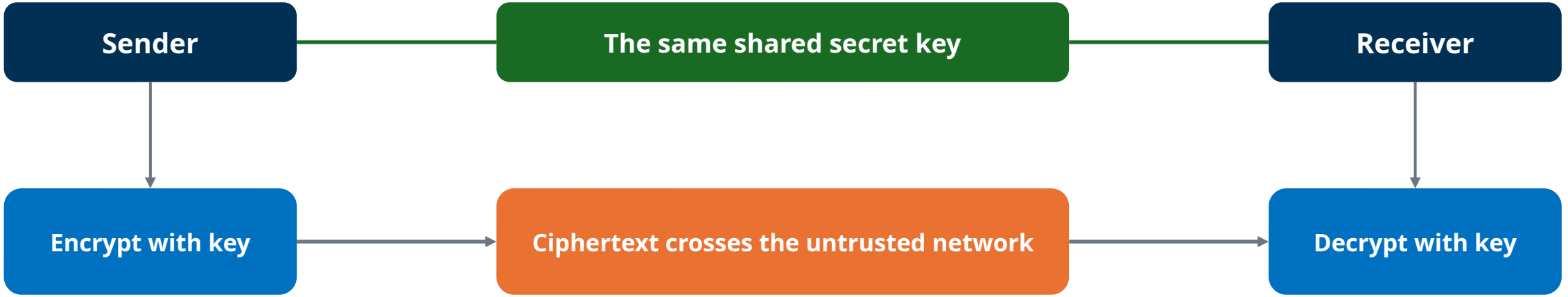
What Carried Forward: Confusion and Diffusion

- In 1949 Claude Shannon named the two properties a strong cipher needs.
- Confusion: make the relationship between the key and the ciphertext as complicated as possible, so that knowing the ciphertext tells you nothing about the key. Substitution provides confusion.
- Diffusion: spread the influence of each plaintext symbol across many ciphertext symbols, so statistical patterns disappear. Transposition and mixing provide diffusion.
- Classical ciphers use one of these once. Modern block ciphers such as AES apply substitution and mixing in repeated rounds, so a single changed input bit changes about half of the output bits. That is the avalanche effect, which returns when we study hash functions.

Symmetric Encryption

One shared key at both ends

The Symmetric Model



- Both ends use one secret key. Anyone holding the key can encrypt and decrypt, so the key must be shared in advance and kept secret from everyone else.
- Examples: DES, 3DES, AES, RC4, ChaCha20. All are very fast, which is why symmetric encryption carries almost all real data.

The Key Distribution Problem

- Problem 1, delivery: how do two parties who have never met agree on a secret key over a network that an attacker can read? Sending the key in the clear defeats the purpose.
- Problem 2, scale: if every pair of users needs its own secret key, the number of keys for n users is n(n-1)/2.

Number of Users	Keys Needed (n(n-1)/2)
10	45
100	4,950
1,000	499,500

- Asymmetric cryptography, later today, exists largely to solve these two problems.

Symmetric Encryption: Strengths, Weaknesses, Use Cases

Strengths

- Very fast, so it suits large volumes of data and high-speed links.
- Efficient in hardware: modern CPUs include AES instructions (AES-NI).
- Short keys give strong security: 128 bits is considered secure for the foreseeable future.
- Simple conceptual model: one key, one operation in each direction.

Weaknesses

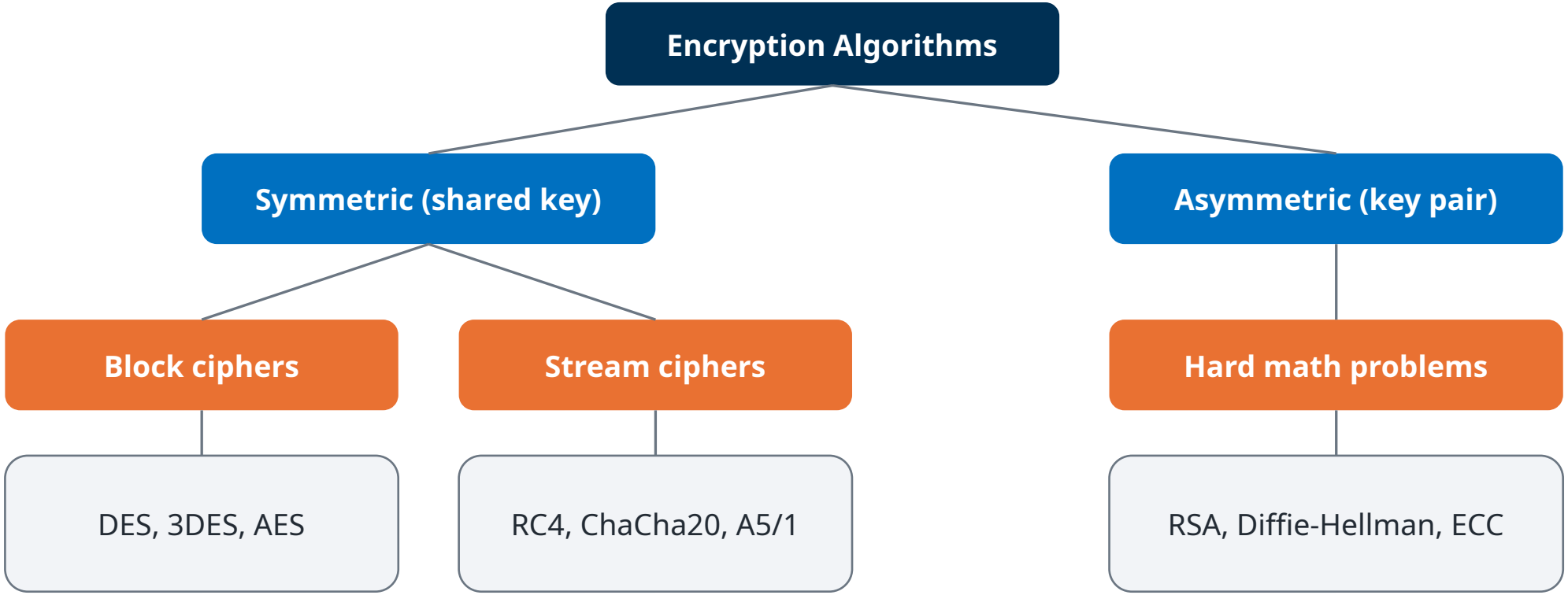
- The key distribution problem: both ends need the secret before communication starts.
- Key count grows as n(n-1)/2, which does not scale.
- Gives no non-repudiation: either holder could have created any message.
- A leaked key exposes everything ever protected with it.

Use cases: bulk data encryption in VPN tunnels, TLS sessions, Wi-Fi (WPA2 and WPA3 use AES), disk and file encryption, and database encryption.

Block vs Stream Ciphers

Two ways to process data with a shared key

Where Block and Stream Ciphers Fit



Two questions classify a cipher: how many keys does it use, and how does it process data. Block versus stream is conventionally a split within symmetric ciphers.

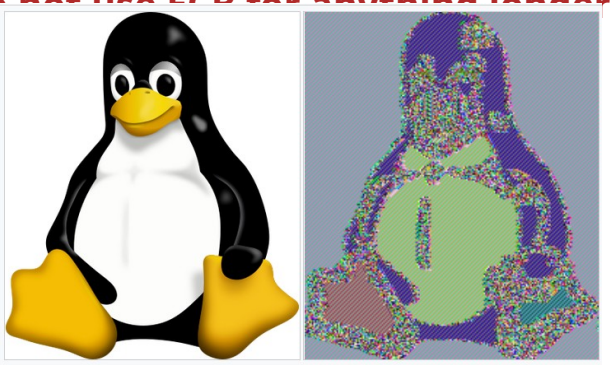
Block Ciphers: Mechanism

- A block cipher encrypts data in fixed-size chunks called blocks, all using the same key. DES and 3DES use 64-bit blocks; AES uses 128-bit blocks.
- Real messages are rarely an exact multiple of the block size, so the final block is padded to full length. The receiver removes the padding after decryption.
- A block cipher on its own only encrypts one block. To encrypt a long message, a mode of operation defines how successive blocks are chained together. The choice of mode matters as much as the choice of cipher.
- Inside the cipher, each block passes through many rounds of substitution and mixing, providing Shannon's confusion and diffusion.

Modes of Operation I: ECB and CBC

ECB: Electronic Codebook

- Every block is encrypted independently with the same key.
- Identical plaintext blocks always give identical ciphertext blocks. Patterns in the data show through.
- The classic demonstration encrypts a bitmap image in ECB mode: the picture is still clearly recognisable in the ciphertext.
- **Verdict: do not use ECB for anything longer than one block.**



Original image

Using ECB allows patterns to be easily discerned

CBC: Cipher Block Chaining

- Each plaintext block is XORed with the previous ciphertext block before it is encrypted. The first block uses a random initialisation vector (IV).
- Identical plaintext blocks now produce different ciphertext, so patterns disappear.
- Costs: encryption is sequential, padding is required, and a bit error spreads into the following block. Careless padding checks have also enabled padding oracle attacks.
- **Verdict: sound if used correctly, but largely superseded.**

Modes of Operation II: CTR and GCM

CTR: Counter Mode

- The block cipher encrypts a running counter, not the message. The output is a keystream, which is XORed with the plaintext.
- This turns a block cipher into a stream cipher. No padding is needed, and blocks can be processed in parallel.
- **Critical rule: never reuse a counter or nonce value with the same key.**

GCM: Galois/Counter Mode

- GCM is CTR mode plus an authentication tag. It provides confidentiality and integrity together, called authenticated encryption.
- Fast, parallelisable, and the default choice in TLS 1.2 and 1.3 and in IPsec ESP.
- The same nonce rule applies: a repeated nonce breaks it badly.

The line between the two families is blurred: CTR and GCM use a block cipher but behave like stream ciphers. This is why the choice of mode is part of the design, not an afterthought.

Block Cipher Algorithms: DES, 3DES, AES

Algorithm	Block Size	Key Size	Status	Notes
DES	64 bits	56 bits	Broken, retired	Key space is small enough to brute force; a purpose-built machine did so in 1998.
3DES	64 bits	Up to 168 bits (about 112 bits of real security)	Deprecated	Applies DES three times. Slow, and the 64-bit block allows birthday attacks on large volumes of data (Sweet32).
AES	128 bits	128, 192, or 256 bits	Current standard	Selected by open NIST competition (Rijndael, standardised 2001, FIPS 197). Fast in hardware and software.

Recall the IPsec framework table in Week 9: this is the confidentiality row, and AES is the choice you would make today.

Block Ciphers: Strengths, Weaknesses, Use Cases

Strengths

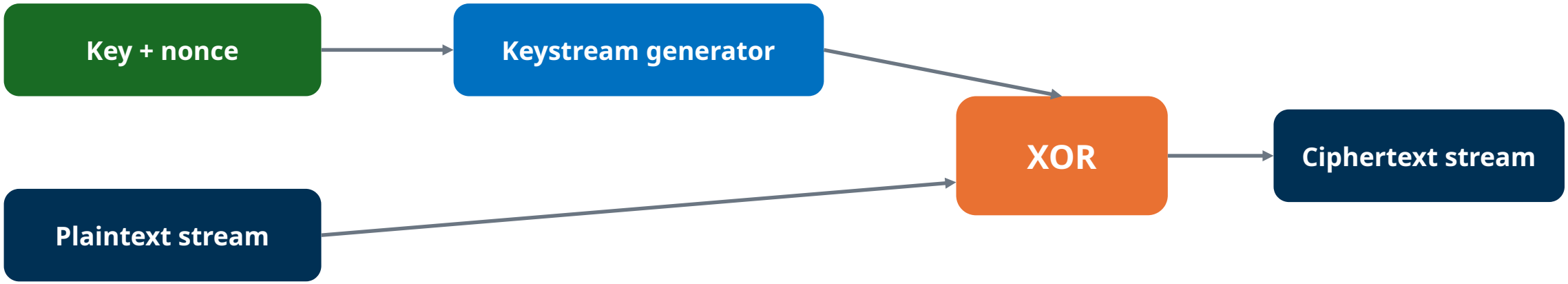
- Very well studied: AES has withstood over two decades of public cryptanalysis.
- Hardware acceleration (AES-NI) makes it extremely fast on most servers and laptops.
- Flexible: different modes suit different needs, from random-access disk encryption to authenticated network traffic.
- GCM gives encryption and integrity in a single operation.

Weaknesses

- Mode choice is easy to get wrong: ECB leaks patterns, and CBC needs careful padding handling.
- Padding adds overhead, and short messages still consume a whole block.
- Nonce or IV mistakes can be catastrophic in CTR and GCM.
- Without hardware support, some block ciphers are slower than modern stream ciphers.

Use cases: full-disk encryption (AES-XTS), file and database encryption, TLS and IPsec bulk data (AES-GCM), Wi-Fi WPA2 and WPA3, and VPN tunnels.

Stream Ciphers: Mechanism



- A stream cipher generates a pseudo-random keystream from the key (and a nonce), then XORs it with the plaintext one bit or byte at a time. Decryption is the same XOR with the same keystream.
- The theoretical ideal is the one-time pad: a truly random key as long as the message, used once. Shannon proved it gives perfect secrecy, but it is impractical because the key must be shared securely and never reused. Stream ciphers approximate it with a short key.

Stream Cipher Algorithms: RC4, ChaCha20, A5/1

Algorithm	Key Size	Status	Where It Appeared	Notes
RC4	40 to 2048 bits	Broken, prohibited in TLS (RFC 7465)	WEP, early WPA (TKIP), SSL and TLS	Statistical biases in the keystream allow plaintext recovery. Simple and fast, which made it popular.
ChaCha20	256 bits	Current, recommended	TLS 1.3 (with Poly1305), WireGuard, SSH	Fast in software without special hardware, which suits mobile devices. Designed by Bernstein.
A5/1	64 bits	Broken in practice	GSM (2G) mobile networks	Short key and published attacks. Newer mobile generations use stronger ciphers.

The Keystream Reuse Weakness: WEP Case Study

- The fatal rule: never use the same keystream twice. If two messages are encrypted with the same keystream, XORing the two ciphertexts cancels the keystream and leaves the XOR of the two plaintexts. An attacker can then recover both using known or guessable content.
- WEP (the original Wi-Fi security) used RC4 with a 24-bit initialisation vector (IV) sent in the clear. A 24-bit IV has only about 16.7 million values, so on a busy network the same IV, and therefore the same keystream, repeats within a few thousand packets.
- WEP also mixed the IV directly into the RC4 key in a way that leaked key material, so attackers could recover the key itself after capturing enough traffic. The attack tools became widely available in the early 2000s.
- Lessons: nonces must never repeat, ciphers must be used in a sound construction, and a secure algorithm inside an insecure protocol is still insecure.

Stream Ciphers: Strengths, Weaknesses, Use Cases

Strengths

- Fast and low latency: each bit or byte can be encrypted as it arrives.
- No padding, and well suited to data of unknown length such as live streams.
- A transmission error damages only the corrupted bit, not the following data.
- Small and simple to implement, including on constrained devices.

Weaknesses

- Keystream reuse is catastrophic and easy to cause by accident.
- On its own offers no integrity: flipping a ciphertext bit flips the same plaintext bit, so a MAC or authenticated mode is required.
- A weak design is hard to spot: RC4 and A5/1 both looked reasonable before they were broken.
- Needs a unique nonce per message, which the protocol must manage.

Use cases: TLS 1.3 on mobile devices (ChaCha20-Poly1305), WireGuard VPN, SSH, live voice and video where low latency matters, and mobile network encryption.

Block vs Stream: Full Comparison

Aspect	Block Cipher	Stream Cipher
Processing unit	Fixed-size blocks (64 or 128 bits)	One bit or byte at a time
Padding	Required for the final block in most modes	Not needed
Speed and latency	Very fast with hardware support; must wait for a full block	Fast in software; low latency
Error propagation	Depends on the mode (CBC spreads an error into the next block)	A single bit error stays a single bit error
Example algorithms	DES, 3DES, AES	RC4, ChaCha20, A5/1
Characteristic weakness	Wrong mode or padding handling	Keystream reuse
Typical use	Disk, file, and bulk network encryption	Streaming, mobile, low-latency links

Scenario: Which Would You Choose?

Scenario	Reasonable Choice	Why
Encrypting a laptop's whole disk	Block cipher: AES in XTS mode	Data is stored in fixed-size sectors and needs random access. Hardware AES makes it nearly free.
A live voice call	Stream-style: AES in counter mode, or ChaCha20	Packets are small and arrive continuously, so low latency and no padding matter. Real voice systems (SRTP) use AES in counter mode.
A TLS web session	Either, with authentication: AES-GCM or ChaCha20-Poly1305	AES-GCM where hardware AES exists; ChaCha20-Poly1305 on phones without it. Both give encryption and integrity.

Notice that the best answer is often a block cipher running in a stream-like mode. The real decision is the construction, not the label.

Asymmetric Encryption

A pair of keys instead of one shared secret

The Key Pair

Public key: shared openly with anyone

- Anyone may have it, so distribution is easy.
- Used to encrypt data for the owner, or to verify the owner's signature.

Private key: never leaves its owner

- Known only to the owner, so it must be protected.
- Used to decrypt data sent to the owner, or to create the owner's signature.

- The two keys are mathematically linked: what one key does, only the other can undo. Working out the private key from the public key is computationally infeasible.
- For confidentiality: encrypt with the receiver's public key, and only the receiver's private key can decrypt. Signatures reverse the roles (private key signs, public key verifies) and are the subject of the next deck.

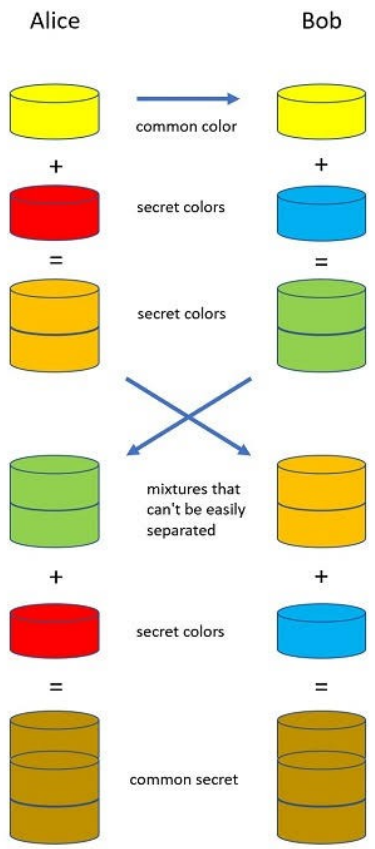
RSA

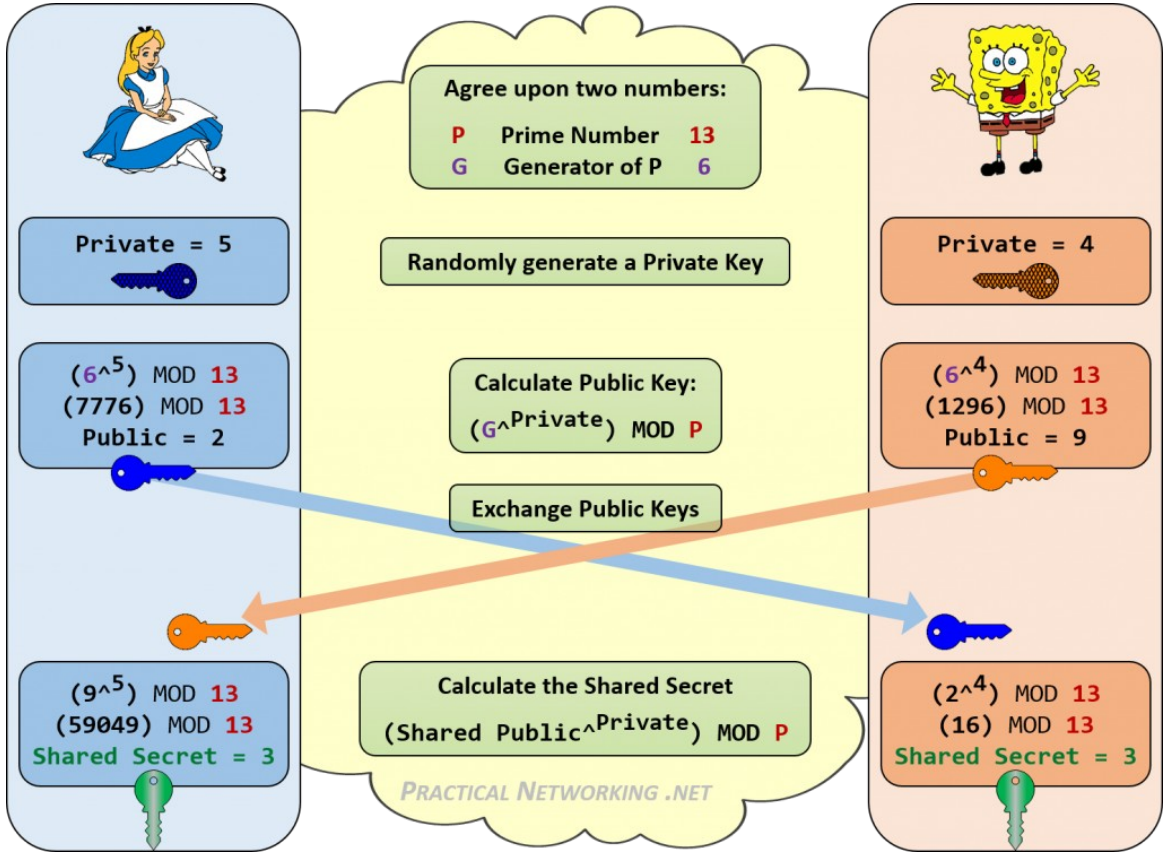
- Mechanism: security rests on the difficulty of factoring the product of two very large prime numbers. Multiplying two primes is easy; recovering them from the product is not.
- Key sizes: 2048 bits is the accepted minimum today, and 3072 bits is recommended for longer-term protection. Keys under 2048 bits should not be used.
- Uses: key transport (encrypting a small symmetric key for the receiver) and digital signatures. RSA encrypts only small values, with a padding scheme such as Optimal Asymmetric Encryption Padding (OAEP), so it is never used to encrypt bulk data directly.
- Status: still widely deployed, but being replaced by elliptic curve methods in new designs, and vulnerable to a large quantum computer.

Diffie-Hellman Key Agreement

- Purpose: two parties who have never met agree on a shared secret over a public channel. Diffie-Hellman does not encrypt anything itself; it produces the shared key that symmetric encryption then uses.
- How it works, in outline: each side keeps a private value and sends a public value computed from it. Each side combines its own private value with the other's public value and arrives at the same shared secret. An eavesdropper sees both public values but cannot compute the secret, because that requires solving the discrete logarithm problem.
- Ephemeral Diffie-Hellman uses fresh values for every session, giving forward secrecy: stealing a long-term key later does not expose past sessions.
- Weakness: plain Diffie-Hellman does not authenticate the other party, so a man in the middle can agree separate secrets with each side. It must be combined with signatures or certificates, which is the topic of the next deck.
- Callback: the Diffie-Hellman group in the IPsec framework table (Week 9) selects the size of the numbers used.

Diffie-Hellman Key Agreement





Elliptic Curve Cryptography (ECC)

- Mechanism: the same style of key pair, but based on the difficulty of the discrete logarithm problem on an elliptic curve instead of on factoring or ordinary modular arithmetic.
- The main advantage is size. A 256-bit elliptic curve key gives roughly the security of a 3072-bit RSA key, so keys, signatures, and computation are all smaller and faster.
- Common forms: ECDH (key agreement) and ECDSA (signatures), using curves such as P-256 and Curve25519.
- This makes ECC especially attractive for mobile devices, smart cards, and IoT, and it is now the default in TLS 1.3 and many VPNs.
- Like RSA and Diffie-Hellman, it would be broken by a large quantum computer. NIST has standardised post-quantum replacements, which are now beginning to be deployed.

RSA vs Diffie-Hellman vs ECC

Aspect	RSA	Diffie-Hellman	ECC
Main purpose	Key transport and signatures	Key agreement	Key agreement (ECDH) and signatures (ECDSA)
Hard problem	Integer factoring	Discrete logarithm	Elliptic curve discrete logarithm
Key size for about 128-bit security	3072 bits	3072 bits	256 bits
Provides forward secrecy?	Not by itself	Yes, when ephemeral	Yes, when ephemeral (ECDHE)
Typical weakness	Large keys, slow	No authentication on its own	Implementation and curve choice

All three share the same long-term threat: a sufficiently large quantum computer.

Asymmetric Encryption: Strengths, Weaknesses, Use Cases

Strengths

- Solves key distribution: the public key can be published openly.
- Scales well: n users need only n key pairs, not n(n-1)/2 shared secrets.
- Enables digital signatures, and therefore authentication and non-repudiation.
- Allows two strangers to establish a secret over an open network.

Weaknesses

- Orders of magnitude slower than symmetric encryption, so unsuitable for bulk data.
- Larger keys and larger outputs.
- You must be sure a public key really belongs to its claimed owner. This is the problem certificates solve.
- RSA, Diffie-Hellman, and ECC are all vulnerable to a future quantum computer.

Use cases: key exchange in TLS and IKE, digital signatures and certificates, SSH authentication, and secure email (S/MIME and PGP).

Symmetric vs Asymmetric, and Hybrid Encryption

Using each where it is strongest

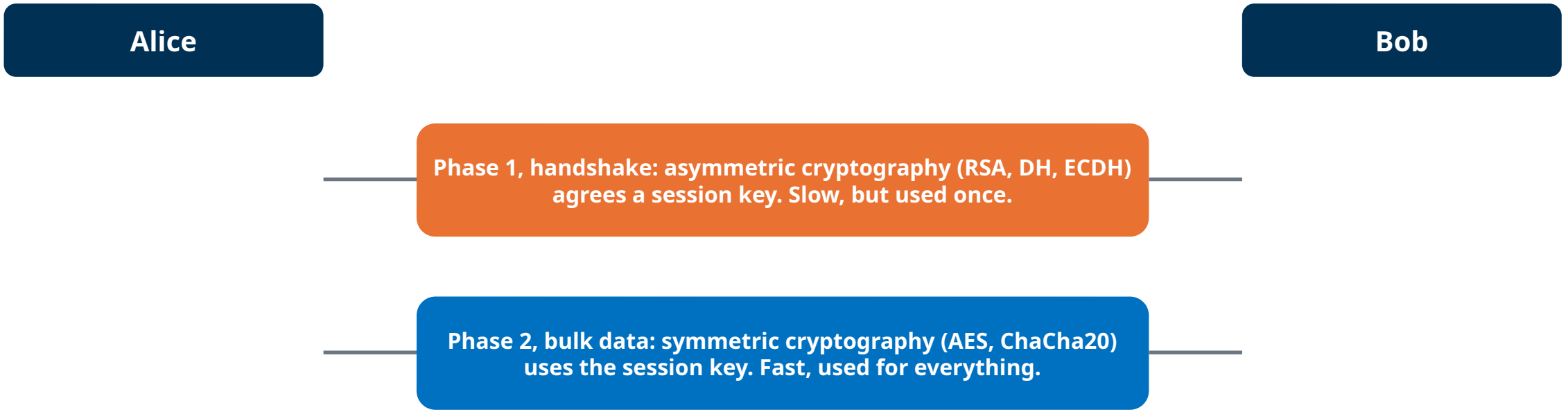
COMPARISON

Symmetric vs Asymmetric: Full Comparison

Aspect	Symmetric	Asymmetric
Keys	One shared secret key	A public and private key pair
Speed	Very fast	Much slower
Key distribution	Difficult: the secret must be shared first	Easy: the public key can be published
Keys needed for n users	n(n-1)/2	n key pairs
Security services	Confidentiality (with a MAC, integrity)	Confidentiality, authentication, non-repudiation (signatures)
Typical key size	128 to 256 bits	256 bits (ECC) to 3072 bits (RSA)
Examples	AES, ChaCha20	RSA, Diffie-Hellman, ECC
Best used for	Encrypting bulk data	Key exchange and signatures

COMPARISON

Hybrid Encryption: The Synthesis



- Asymmetric cryptography solves distribution but is slow. Symmetric cryptography is fast but cannot distribute its own keys. Hybrid encryption uses each for the job it does well.
- The session key is temporary, so each connection gets a fresh key, limiting the damage if one is ever exposed.

COMPARISON

Worked Example: TLS and IPsec Are Hybrid Systems

Protocol	Asymmetric Part (setup)	Symmetric Part (data)
TLS 1.3 (web, SSL VPN)	ECDHE key agreement, authenticated with a certificate and signature	AES-GCM or ChaCha20-Poly1305 for all application data
IPsec (IKE, then ESP)	IKE uses a Diffie-Hellman exchange and authentication (pre-shared key or certificates) to build the keys	ESP encrypts traffic with AES, often in GCM mode
SSH	Key exchange (such as Curve25519) and host and user authentication	AES or ChaCha20 for the session

In every case the expensive asymmetric step happens once at the start, then fast symmetric encryption carries the data.

Summary

- Cryptography provides confidentiality, integrity, authentication, and non-repudiation. Encryption provides the first of these; the next deck covers the rest.
- Kerckhoffs's principle: the algorithm is public and the key is the secret. Classical ciphers failed on key space and frequency patterns, which taught the principles of confusion and diffusion.
- Block ciphers (AES) encrypt fixed blocks and depend on the mode of operation. Stream ciphers (ChaCha20) encrypt a keystream and depend on never reusing a nonce. CTR and GCM blur the line between them.
- Asymmetric cryptography (RSA, Diffie-Hellman, ECC) solves key distribution but is slow. Hybrid encryption combines both, and that is how TLS, IKE, and SSH work.

Coming Up

- Next deck: integrity, signatures, and PKI. Hash functions, MACs and HMAC, digital signatures, X.509 certificates, certificate authorities, and a full TLS handshake walkthrough.
- One open question from today carries forward: when you receive a public key, how do you know it belongs to who it claims to? Diffie-Hellman without authentication is open to a man in the middle. Certificates are the answer.
- Tutorial worksheet: study-notes questions covering today's material, for the mid-semester test and final exam.

References

- Stallings, Cryptography and Network Security: Principles and Practice, 8th Ed., Pearson.
- Santos and Stuppi, CCNA Security 210-260 Official Cert Guide, Cisco Press.
- NIST FIPS 197, Advanced Encryption Standard (AES).
- IETF RFC 8446, The Transport Layer Security (TLS) Protocol Version 1.3.
- IETF RFC 7465, Prohibiting RC4 Cipher Suites.